PROJECT SYNOPSIS

ON

"DecoStone Material Management System (MMS)"

- Submitted By -

Parth Sanjay Morabale	**(24213440464)**
Ayush Rajesh Patel	**(24213440466)**
Samarth Dadaso Patil	**(24213440479)**
Suyash Sunil Shalbidre	**(24213440483)**

Under The Guidance Of

Ms. Supriya Dongre.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SANJAY GHODAWAT INSTITUTE, ATIGRE

Academic Year: 2026-27

Certificate

This is to certify that the project synopsis for the work entitled

"DecoStone Material Management System (MMS)"

has been prepared and submitted by

Parth Sanjay Morabale	**(24213440464)**
Ayush Rajesh Patel	**(24213440466)**
Samarth Dadaso Patil	**(24213440479)**
Suyash Sunil Shalbidre	**(24213440483)**

In partial fulfillment of the requirements for the

Diploma in Computer Science & Engineering

Maharashtra State Board of Technical Education

During the academic year 2026-27 under the guidance of

Project Guide	H.O.D	Director
(Ms. S. S. Dongre)	(Mr. A. D. Patil)	(Dr. V. V. Giri)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACKNOWLEDGEMENT

We express our sincere gratitude to Sanjay Ghodawat Institute, Atigre, and the Department of Computer Science & Engineering for academic support and resources for this project synopsis.

We thank our project guide, Ms. Supriya Dongre, for her guidance and feedback. We also thank faculty members, staff and peers who helped us understand material inventory and warehouse workflows.

This synopsis has been prepared by the four student authors listed below. The guide and department may review the technical description and completion records before certification.

Student Name	Enrollment Number
Parth Sanjay Morabale	24213440464
Ayush Rajesh Patel	24213440466
Samarth Dadaso Patil	24213440479
Suyash Sunil Shalbidre	24213440483

Date: _____
Place: Atigre

ABSTRACT

DecoStone Material Management System (MMS) is a web-based inventory application for businesses handling tiles, marble, granite and related building materials across multiple godowns. It brings material and category records, warehouse stock, receipts, dispatches, transfers and adjustments into a shared workflow. The system uses a React single-page frontend, an Express REST backend and a relational SQLite database. Role-based access supports administrator, manager and supervisor workflows, including supervisors assigned to multiple godowns. Dashboard summaries, low-stock visibility, movement history and report exports help users review inventory activity. This synopsis presents the problem, related work, requirements, architecture, data model, modules, security considerations and validation plan. The team should verify transaction correctness, access boundaries, responsive operation and report accuracy before final demonstration.

Keywords: Inventory management, warehouse, godown, stock movement, SQLite, role-based access.

INDEX

Chapter No.	Title	Page No.
1	Introduction	7
2	Literature Survey	8
3	Problem Definition	11
4	Existing System	12
5	Proposed System	13
6	Objectives and Scope	14
7	Requirements and Feasibility	15
8	Methodology and Architecture	16
9	Database Design	21
10	System Modules and Workflows	22
11	Security and Access Control	23
12	Testing and Validation Plan	24
13	Resources and Consumables	25
14	Conclusion	26
R	References and Bibliography	27
A	Action Plan	28

LIST OF TABLES

Table No.	Description	Page No.
1	Literature comparison	9
2	Functional requirements	15
3	Core data entities	21
4	Application modules	22
5	Validation checklist	24
6	Development resources	25
7	Project action plan	28

LIST OF FIGURES

Figure No.	Description	Page No.
1	DecoStone MMS architecture and request-response flow	16

LIST OF GRAPHS

1. Stock movement timeline 17
2. Material stock levels 18
3. Stock by godown..... 19
4. Current stock by category 20

1. INTRODUCTION

Building-material businesses manage varied inventory such as tiles, sanitaryware, stone slabs, adhesives and related products. A material may be stored in more than one godown, measured in a product-specific unit, and moved through receipts, issues, transfers and stock adjustments. Staff need to know what is available, where it is stored and how each balance changed.

When these events are recorded in separate registers or spreadsheets, the same item may have inconsistent balances. Delayed updates make it harder to prevent stockouts, validate transfers or trace adjustments. A shared application can centralize material definitions and warehouse balances while retaining records of stock movements.

DecoStone MMS provides a browser-based workspace for managing these records. The application separates the React frontend, Express API and SQLite persistence. Users work through role-specific screens while the server validates requests and stores inventory changes. This synopsis describes the project design; final implementation status and unresolved defects should be confirmed during the demonstration review.

1.1 Relevance and Rationale

Inventory control is an operational and supply-chain concern [1]. Warehouse information systems also connect storage operations to organizational capabilities and performance [3]. This project focuses on practical functions for a multi-godown material business: current balances, movement records, low-stock review and controlled access.

1.2 Synopsis Organization

The report reviews prior work, defines the problem, compares the existing approach with the proposed application, states objectives and scope, and explains requirements, architecture, data design, workflows, access control, validation and resources. References and an action plan conclude the report.

2. LITERATURE SURVEY

This focused literature survey reviews work on inventory control, warehouse management systems, item classification, forecasting and intelligent warehouses. The studies address different decision levels; together they inform the project design and help define what the present system does not claim to provide.

2.1 Inventory Management

Munyaka and Yadavalli review inventory-management concepts and implementations across different demand conditions and organizational objectives [1]. Their review shows why inventory policies must match the operating context. For DecoStone MMS, the practical implication is to maintain dependable item and location balances and preserve the movement history used to explain them.

Ramanathan proposes a weighted linear-optimization approach to multi-criteria ABC inventory classification [2]. It ranks items using several criteria rather than relying on a single measure. This is relevant as a possible future prioritization method, but the current application records stock and thresholds; it does not calculate ABC classes or implement the optimization model.

2.2 Warehouse Systems and Forecasting

Autry, Griffis, Goldsby and Bobbitt study warehouse-management systems in relation to firms' resource commitments, capabilities and organizational performance [3]. Their work places WMS adoption within operational and organizational practice, rather than treating software alone as the outcome. This project therefore connects balances to receiving, dispatch and transfer workflows, while recognizing that user training and consistent procedures also matter.

Goltsos and co-authors review the relationship between inventory control and demand forecasting and identify gaps between the two research streams [4]. Forecast estimates are useful only when they inform replenishment decisions. DecoStone currently provides transaction history and threshold-based stock alerts; it does not forecast demand or automate replenishment.

2.3 Intelligent Warehouses

Tubis and Rohman systematically review intelligent warehouses in the Industry 4.0 context, covering technology areas such as IoT, visual systems, artificial intelligence and automation [5]. These approaches depend on infrastructure beyond a browser-based inventory application. They provide a future direction for the project, not a feature claimed in the current implementation.

Mashayekhy and co-authors survey research on the use of IoT in inventory management and discuss its applications and research opportunities [6]. The present system records user-entered transactions and does not receive sensor or RFID readings. This distinction keeps the project's current scope clear.

2.4 Comparative Findings

Study	Research focus	Relevance to this project
Munyaka & Yadavalli [1]	Review of inventory control and demand conditions	Keep stock balances and control rules explicit.
Ramanathan [2]	Multi-criteria ABC classification	Potential future prioritization; not implemented here.
Autry et al. [3]	WMS resources, capabilities and performance	Connect stock balances to real warehouse workflows.
Goltsos et al. [4]	Relationship between forecasting and inventory control	Keep threshold alerts distinct from demand forecasts.
Tubis & Rohman [5]	Intelligent warehouses and Industry 4.0 technologies	Treat IoT and automation as future scope.
Mashayekhy et al. [6]	IoT applications in inventory management	Current stock changes remain transaction-based.

The studies operate at complementary levels. Munyaka and Yadavalli review inventory-control choices [1]; Ramanathan develops a specific item-classification method [2]; Autry and co-authors examine WMS capabilities in organizations [3]; Goltsos and co-authors analyze the link between forecasts and inventory decisions [4]; and the warehouse/IoT surveys examine technology-enabled sensing and automation [5], [6]. DecoStone focuses on the transactional foundation—material records, godown balances, controlled stock movements and traceable history—without claiming the optimization, forecasting or automation addressed by the other work.

2.5 Research Gap and Project Position

This project does not claim that multi-location inventory systems are absent from prior research or commercial practice. Its project-level need is more specific: provide a straightforward, role-scoped way for a building-material business to connect material records with stock at each godown and retain a clear history of changes. The reviewed work supports the importance of inventory control and warehouse information, while the proposed implementation is deliberately narrower than an enterprise WMS or an optimization platform.

The design responds to that need by treating a balance as a material–godown pair, recording receipts, dispatches, transfers and adjustments, and limiting user access according to assigned responsibilities. These choices translate the literature into requirements for transaction consistency, location-level visibility and traceability. They are design goals that must be confirmed through the validation plan, not results inferred from the literature.

Future work could evaluate demand-based replenishment or ABC prioritization after the team has collected sufficiently complete and reliable transaction data. Any such extension should define its input criteria, compare its recommendations with a baseline and report measured results. Forecasting and classification are not presented as current project capabilities.

3. PROBLEM DEFINITION

The project addresses the difficulty of maintaining a consistent, timely and auditable view of building-material stock across multiple godowns. A business needs to identify each material, category and unit; record receipts and dispatches; transfer quantities between locations; and determine whether each godown has sufficient stock.

The problem is not simply the absence of a digital list. Inventory changes must preserve a correct balance and explain how that balance was reached. A transfer must decrease stock at its source and increase stock at its destination exactly once. Access should reflect responsibility: an administrator manages the system, managers perform permitted operations, and supervisors may be assigned to one or more godowns.

3.1 Problem Statement

To design and implement a web-based material management system that maintains material and category records, records stock transactions for multiple godowns, provides role-scoped access and presents useful stock and movement information while preserving transaction history.

3.2 Constraints

The system depends on a reachable server and database, accurate user-entered transactions, correct master data and configured access rights. SQLite suits a compact deployment, but high-concurrency or multi-site expansion may require a different database architecture. Hardware sensors, automated purchasing and predictive replenishment are outside the current scope.

4. EXISTING SYSTEM

In a manual or spreadsheet workflow, material descriptions, quantities and storage locations may be maintained in separate files. Staff update a stock sheet after a delivery or dispatch while another person records the event in an order or ledger. Such systems are familiar and inexpensive to start, but rely on consistent procedures and timely manual updates.

4.1 Limitations

- Warehouse balances may become inconsistent when movements are late or duplicated.
- A transfer may appear as an issue without a matching receipt at the destination.
- Historical evidence can be difficult to connect to current material or category records.
- Finding out-of-stock items can require repeated checks across separate lists.
- User responsibilities and warehouse visibility may not be enforced consistently.
- Preparing summaries and exports may require manual filtering and reformatting.

4.2 Improvement Opportunity

A centralized application can validate a transaction at entry, associate it with a material and godown, and update related records in one server-side workflow. Consistent role checks, movement history and reports can improve reviewability. These benefits depend on correct implementation and disciplined data entry.

5. PROPOSED SYSTEM

The proposed DecoStone MMS is a browser-based application with three layers. The React frontend presents role-specific pages and sends requests to an Express REST API. The API authenticates users, validates operations and applies business rules. SQLite stores users, materials, categories, godowns, balances and transaction records.

5.1 Main Functions

- Manage material and category records, including edits and controlled deletion or archival where history is linked.
- Maintain godown records and assign supervisors to multiple godowns where authorized.
- Record stock receipts, dispatches, transfers and adjustments with quantities and transaction context.
- Review stock by material and godown, including low-stock and out-of-stock conditions.
- View movement history, audit activity, dashboard summaries and warehouse/category reports.
- Export supported reports to spreadsheet or PDF formats.

5.2 Operating Principles

Stock-changing operations should be atomic: either all database changes for a transaction succeed or none do. A transfer should link its outgoing and incoming legs and prevent an invalid or duplicate quantity change. Deletion should preserve referential integrity and transaction history. Reports should use the same stored balances and transaction records as the application.

6. OBJECTIVES AND SCOPE

6.1 Objectives

- Create a shared catalogue of materials and categories with units and stock thresholds.
- Represent inventory separately for each active godown.
- Record receipts, dispatches, inter-godown transfers and adjustments.
- Provide secure login and role-based authorization.
- Allow a supervisor to access multiple assigned godowns with server-side scope checks.
- Display stock position, low-stock conditions and movement history.
- Generate readable summaries and downloadable reports.
- Maintain auditability and protect historical references when master records change.

6.2 Scope

The project covers master data, warehouse stock, movements, user access, dashboards, notifications, reports and transaction/audit history represented in the application. Roles include administrator, manager and supervisor. A supervisor can be associated with multiple godowns, subject to backend authorization.

Barcode/RFID scanning, IoT sensors, automatic demand prediction, payment processing, external ERP integration and production cloud availability are not claimed. Future work may evaluate these features after core transaction correctness and deployment requirements are validated.

7. REQUIREMENTS AND FEASIBILITY

7.1 Functional Requirements

ID	Requirement
FR-01	Authenticate users and enforce role permissions.
FR-02	Maintain materials and categories while protecting linked history.
FR-03	Maintain godowns and user-to-godown assignments.
FR-04	Record valid receipts, dispatches, transfers and adjustments.
FR-05	Display stock by material/godown and identify threshold conditions.
FR-06	Show transaction history, audit events, dashboard metrics and reports.
FR-07	Export supported report views.

7.2 Non-Functional Requirements

Usability: common tasks need clear labels and validation feedback. Integrity: related stock changes must remain consistent. Security: credentials must be protected and the server must enforce access rules. Maintainability: interface, API and persistence concerns should be organized. Compatibility: the interface should support common desktop and mobile browsers. Reliability and performance targets must be defined and evaluated under the intended load.

7.3 Feasibility

Technical feasibility is supported by the JavaScript stack and local relational database. Operational feasibility depends on users entering movements promptly and administrators configuring roles correctly. Open-source frameworks reduce licensing costs, while hosting, backups and maintenance still have operational costs. Schedule feasibility should be reviewed against remaining project milestones.

8. METHODOLOGY AND ARCHITECTURE

8.1 Development Method

Parth Sanjay Morabale and Ayush Rajesh Patel worked on the frontend and backend across all modules. Suyash Sunil Shalbidre and Samarth Dadaso Patil handled database design across all modules. Work progressed iteratively from requirements and data modeling through implementation, integration and review.

8.2 Application Layers

Layer	Responsibility
User interface	React screens, forms, tables, navigation, feedback and visual summaries.
API and rules	Express routes, authentication, input validation, authorization and stock operation logic.
Persistence	SQLite tables, constraints, initialization and transaction history.
Reporting	Queries aggregate balances and movements for dashboards and exports.

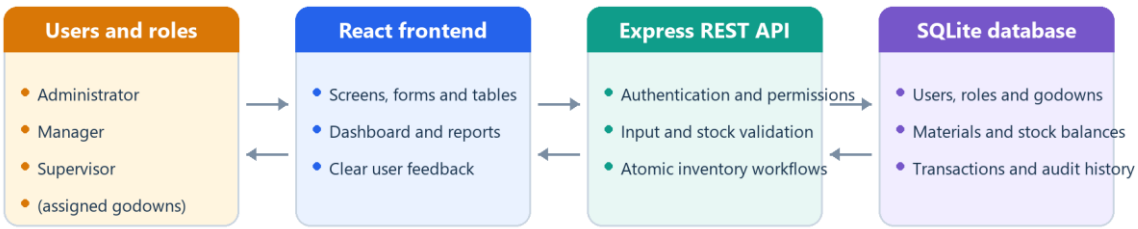


Figure 1. DecoStone MMS architecture and request-response flow

A user submits a request through the interface. The API verifies identity and role, validates identifiers and quantities, and reads or changes the database. A successful stock movement should store its transaction record and update the affected balance consistently. The response returns a clear result for the interface.

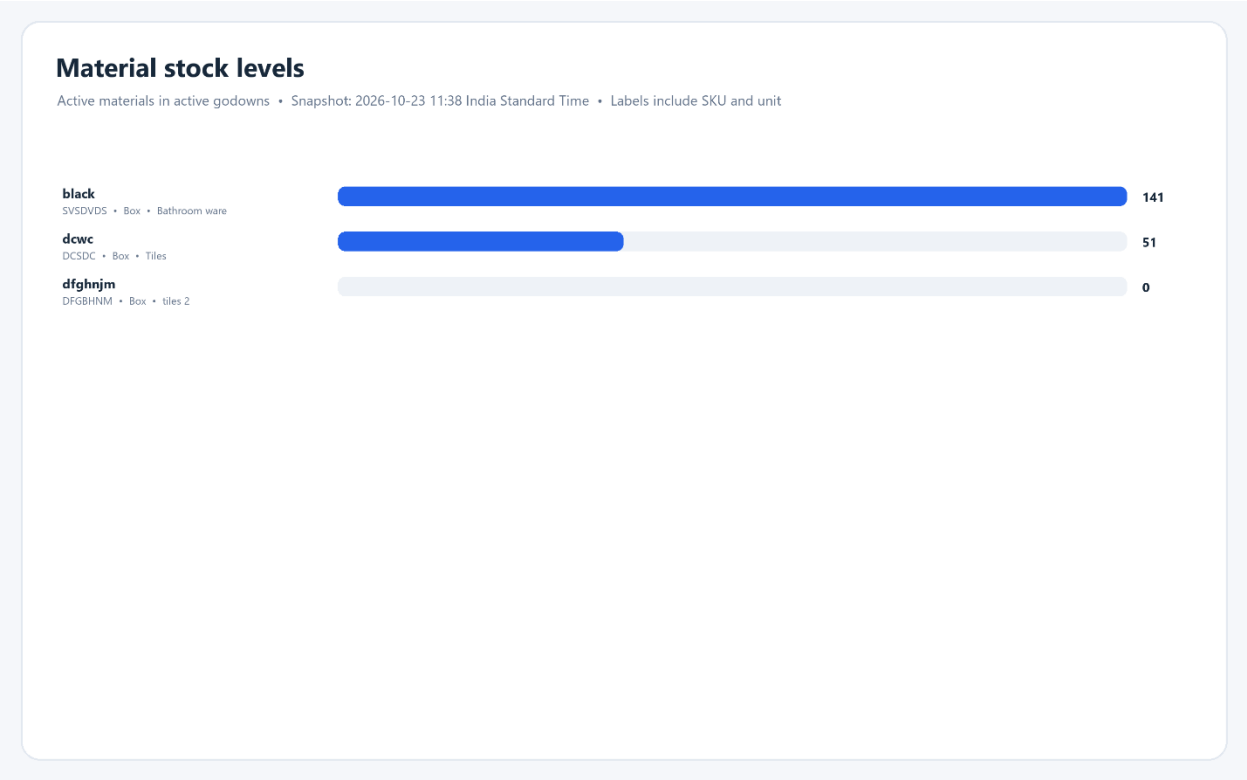
8.3 Transaction Workflow

For a receipt, the server validates the material, destination godown and positive quantity, then records the receipt and updates that balance. For dispatch, it checks sufficient stock before reduction. For transfer, it validates source and destination, checks stock, and commits the paired movement as one operation. An adjustment requires a reason and preserves the prior history.

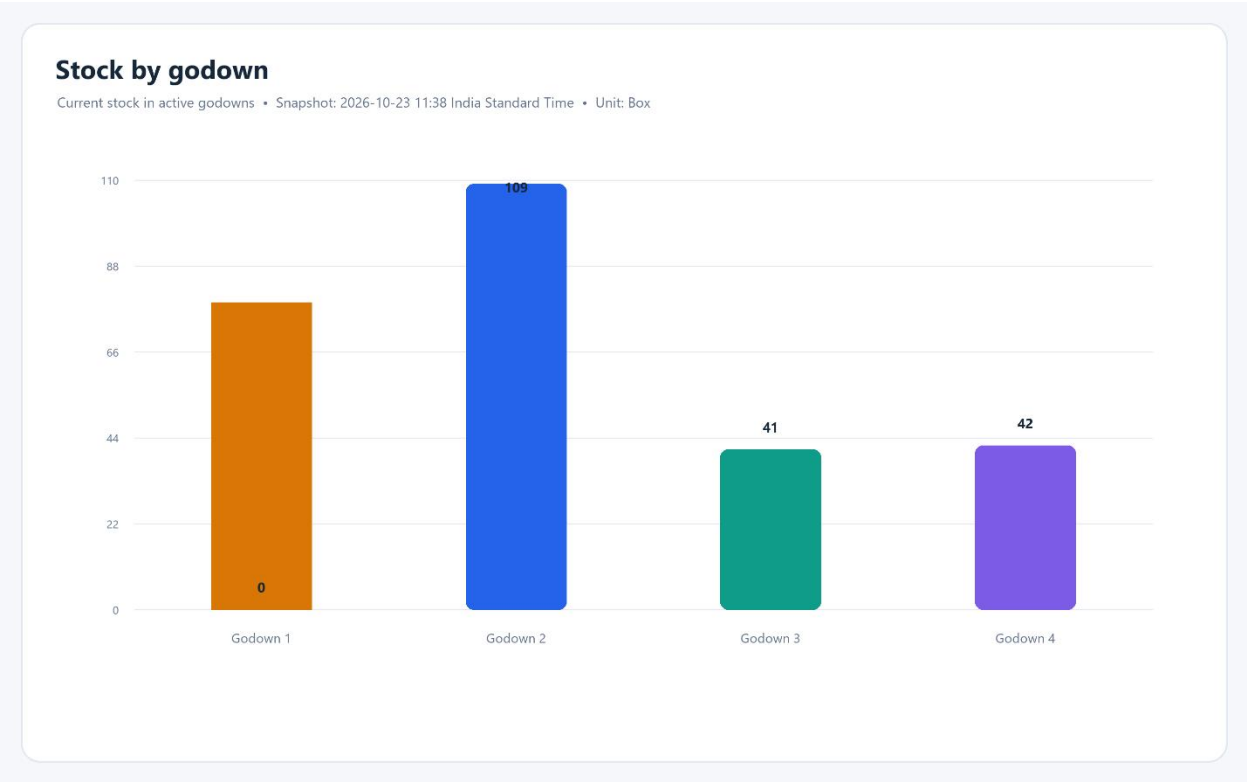
8.4 Stock Movement Timeline



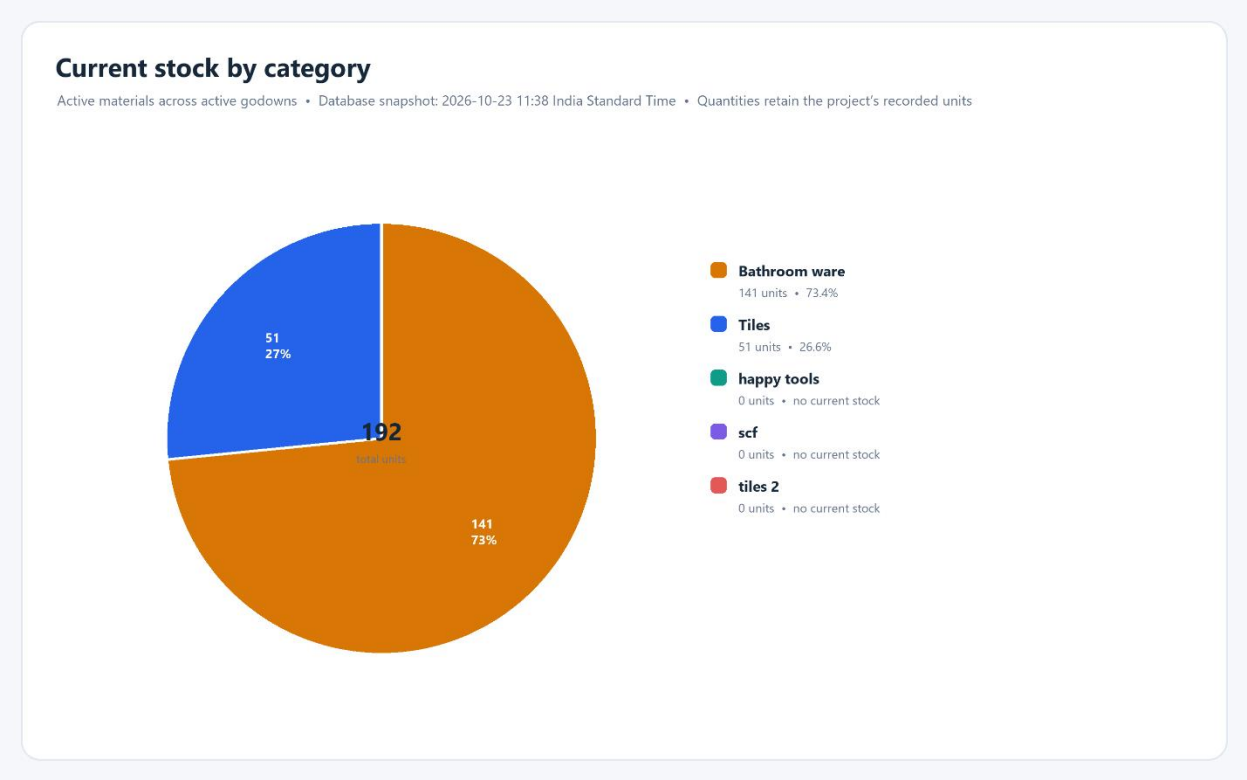
8.5 Material Stock Levels



8.6 Stock By Godown



8.7 Current Stock By Category



9. DATABASE DESIGN

The application uses relational data to represent users, roles, godowns, materials, categories, stock and inventory events. Foreign keys connect operational records to master data. Unique constraints and validation rules should prevent duplicate identifiers and invalid references.

Entity	Purpose	Relationships
Users / roles	Login identity and authorization	User role and godown assignments
Godowns	Physical storage locations	Referenced by stock and movements
Categories	Material classification	One category may classify many materials
Materials	SKU, description, unit and thresholds	Belongs to category; referenced by stock/events
Stock balances	Current quantity by material and godown	Unique material + godown pair
Transactions	Receipt, issue, transfer or adjustment	References material, location(s), user and quantity
Audit events	Trace significant actions	References actor and affected record where available

9.1 Integrity Rules

- A balance is identified by both material and godown; a combined total is not a location balance.
- Quantities must be numeric and valid for the material unit and operation.
- Dispatch and transfer must not create negative stock unless an approved rule permits it.
- A transfer must produce a linked source decrease and destination increase exactly once.
- Delete/archive actions must preserve history and avoid orphaned records.
- Database transactions should group multi-record stock changes atomically [7].

10. SYSTEM MODULES AND WORKFLOWS

The interface and server are organized around the following modules. Route coverage and edge cases should be confirmed through code review and validation before final presentation.

Module	Primary workflow
Authentication and users	Sign in, manage members and grant permitted access.
Godowns	View locations and assign supervisors to one or more godowns.
Categories and materials	Maintain classification, SKU/material details, units and thresholds.
Stock received	Record receipt context and increase a godown balance.
Stock dispatched	Record issue context and reduce available stock.
Transfers	Move quantity between godowns with linked history.
Adjustments and history	Record a reviewed discrepancy with reason and audit trail.
Dashboard and low stock	Review balances, threshold items and recent movements.
Reports and exports	Review warehouse/category data and export supported formats.

10.1 End-to-End Example

An authorized user selects a material and receiving godown, enters a valid quantity and submits a receipt. The server validates and stores the event with its actor and time, then updates that material's balance in that godown. A later transfer selects source and destination; the source must have enough stock. History should show the quantity leaving one godown and arriving at the other without double-counting.

11. SECURITY AND ACCESS CONTROL

The project separates user identity from permissions. The administrator creates members and grants access; operational roles should not gain administrative powers through interface visibility alone. Backend handlers must enforce authorization on protected operations.

11.1 Authentication

The backend dependency manifest includes bcryptjs and JSON Web Token packages. Passwords should be stored only as salted hashes, never plaintext. Tokens should be validated on protected requests and handled with appropriate expiry and secure storage. OWASP guidance describes established controls for authentication and password storage.

11.2 Authorization and Data Scope

A supervisor may be assigned to multiple godowns. Authorization should check that a requested godown is among that supervisor's assignments for each relevant read and write. Client-side filtering is not sufficient. User management, transaction changes and exports should be limited to roles with explicit permission.

11.3 Audit and Data Protection

Movements and significant administrative actions should retain actor, timestamp, operation, quantity, location and reference information. Backup and restore procedures should be documented and tested. Production deployment should use HTTPS, protect environment secrets, restrict database-file access and verify restoration before relying on backups.

12. TESTING AND VALIDATION PLAN

This chapter defines validation work and does not claim every check has already passed. The team should record actual results, evidence and corrective actions in its diary. Test the interface, API and database using realistic multi-godown workflows.

Area	Validation examples
Master data	Create, edit, search and remove unused records; preserve linked history.
Receipt / dispatch	Positive quantity, invalid input, insufficient stock and repeated submission.
Transfer	Source/destination checks; source decrease equals destination increase.
Access control	Role permissions and supervisor access only to assigned godowns.
Reports	Compare totals with database balances and movement records.
Database integrity	Foreign keys, uniqueness, rollback and backup restoration.
Responsive UI	Forms, navigation, tables and dialogs at phone/tablet/desktop widths.
Build and security	Production build, backend configuration and authentication failures.

12.1 Acceptance Conditions

A receipt increases exactly one selected material/godown balance. Dispatch reduces stock only when valid. Transfer reduces the source by quantity q and raises the destination by q, preserving the total. Unauthorized users cannot access restricted routes. Report totals match the database state. These conditions should be evidenced with repeatable tests and recorded results.

13. RESOURCES AND CONSUMABLES

Resource	Purpose
Development computer and browser	Run and review the application locally.
Node.js and npm	Run server and build scripts.
React and Vite	Build the client interface.
Express.js	Implement the backend API.
SQLite / better-sqlite3	Persist relational inventory and transaction data.
Recharts, XLSX, jsPDF	Present charts and export reports.
Git repository	Track changes and collaborate across modules.
Representative test data	Validate workflows using non-sensitive records.

The current software scope requires no physical consumables or warehouse sensors. It does require a backup location, test data and a reliable runtime. Package versions should be confirmed against lockfiles at submission time because dependencies may change.

14. CONCLUSION

DecoStone MMS addresses the need to maintain material and warehouse stock records through a shared web application. Its intended value comes from connecting master data, godown-level balances and auditable stock movements, then presenting the information through alerts, dashboards and reports. The frontend/API/relational database structure separates user interaction from persistence rules.

The key correctness requirement is that each stock operation has one validated, traceable effect on the right material and godown. Role checks must enforce intended boundaries, including supervisors assigned to multiple godowns. The team should demonstrate these properties using repeatable scenarios and database evidence and complete guide-reviewed diary entries before submission.

Future enhancements may include barcode scanning, supplier integration, forecasting, multi-branch deployment and richer analysis. These should follow verification of transaction integrity, backups, access control and report consistency.

REFERENCES AND BIBLIOGRAPHY

[1] J.-C. B. Munyaka and V. S. S. Yadavalli, "Inventory management concepts and implementations: A systematic review," South African Journal of Industrial Engineering, vol. 33, no. 2, pp. 15-36, 2022. doi: https://doi.org/10.7166/33-2-2527.

[2] R. Ramanathan, "ABC inventory classification with multiple-criteria using weighted linear optimization," Computers & Operations Research, vol. 33, no. 3, pp. 695-700, 2006. doi: https://doi.org/10.1016/j.cor.2004.07.014.

[3] C. W. Autry, S. E. Griffis, T. J. Goldsby, and L. M. Bobbitt, "Warehouse management systems: Resource commitment, capabilities, and organizational performance," Journal of Business Logistics, vol. 26, no. 2, pp. 165-183, 2005. doi: https://doi.org/10.1002/j.2158-1592.2005.tb00210.x.

[4] T. E. Goltsos, A. A. Syntetos, C. H. Glock, and G. Ioannou, "Inventory-forecasting: Mind the gap," European Journal of Operational Research, vol. 299, no. 2, pp. 397-419, 2022. doi: https://doi.org/10.1016/j.ejor.2021.07.040.

[5] A. A. Tubis and J. Rohman, "Intelligent warehouse in Industry 4.0: Systematic literature review," Sensors, vol. 23, no. 8, art. 4105, 2023. doi: https://doi.org/10.3390/s23084105.

[6] Y. Mashayekhy, A. Babaei, X.-M. Yuan, and A. Xue, "Impact of Internet of Things (IoT) on inventory management: A literature survey," Logistics, vol. 6, no. 2, art. 33, 2022. doi: https://doi.org/10.3390/logistics6020033.

[7] SQLite Consortium, “Transactions,” SQLite Documentation. [Online]. Available: https://www.sqlite.org/lang_transaction.html. Accessed: 23-Oct-2026.

[8] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Accessed: 23-Oct-2026.

[9] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accessed: 23-Oct-2026.

[10] Meta Platforms, “Quick Start,” React Documentation. [Online]. Available: https://react.dev/learn. Accessed: 23-Oct-2026.

[11] OpenJS Foundation, “Express Documentation.” [Online]. Available: https://expressjs.com/. Accessed: 23-Oct-2026.

[12] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/docs/latest/api/. Accessed: 23-Oct-2026

ACTION PLAN

Scheduled dates	Planned activities
10/09/2026 – 16/09/2026	Confirm domain, team roles and inventory problem; define project scope.
17/09/2026 – 23/09/2026	Gather requirements; map user roles, godown workflows and feasibility.
24/09/2026 – 30/09/2026	Review inventory literature; draft requirements and data entities.
01/10/2026 – 07/10/2026	Plan architecture, database schema, API structure and interface modules.
08/10/2026 – 14/10/2026	Plan catalogue CRUD and stock receipt, dispatch, transfer and adjustment flows.
15/10/2026 – 21/10/2026	Plan integration of roles, multi-godown scope, alerts, audit/history, reports and responsive layouts.
22/10/2026 – 23/10/2026	Plan code/build review, demo flow, synopsis and diary completion.

The plan records scheduled activities only. Actual completion dates, results, attendance and guide review must be entered by the team and verified by the project guide. The presentation and project demo are scheduled for 24 October 2026.